

Rapport LAB 3 BIG DATA

L'objectif de cette partie est de déterminer le mot le plus fréquent du jeu de données en réutilisant la sortie du job MapReduce word_count réalisé précédemment.

L'idée MapReduce pour most_frequent :

- Mapper : lit word\tcount et émet une clé constante avec une valeur qui contient count + word.
- Reducer : reçoit toutes les valeurs sous "MAX" et garde la paire avec le plus grand count (tie-break optionnel).

L'entrée de ce job correspond au répertoire HDFS :

/education/ece_2026_spring_gr01/l.chloe-ece/lab3/word-count

On vient créer le dossier most_frequent et les fichiers mapper et reducer avec -mkdir

Implementation du mapper:

On vient faire vim most_frequent/mapper afin de pouvoir y implémenter le code

import sys

Cette ligne importe le module sys afin de lire les données depuis l'entrée standard : transmission de données au mapper.

def write_kv_array(kv_array):

for kv in kv_array:

print(f'{kv["key"]}\t{kv["value"]}')

Fonction d'affichage d'un couple clé-valeur dans le format attendu par Hadoop : clé puis tab puis valeur.

def read_word_count_lines(text):

key_values = []

Cette fonction permet de traiter les lignes d'entrées issues de word count. Initialisation d'une liste de stockage des couples clé-valeur.

for line in text:

line = line.strip()

if not line:

continue

Chaque ligne est nettoyée pour supprimer les caractères inutiles et les lignes vides sont ignorées

parts = line.split('\t')

if len(parts) != 2:

continue

Découpage de la ligne grâce au caractère de tabulation « \t », si le bon format n'est pas respecté, on ignore la ligne.

word, count_str = parts[0], parts[1]

On vient extraire chaque mot et son nombre d'occurrence.

try:

count = int(count_str)

except ValueError:

continue

On convertit le nombre d'occurrence au format entier

key_values.append({'key': 'MAX', 'value': f'{count}\t{word}'})

On crée une clé constante MAX, qui permet d'enregistrer les données, on utilisera la même clé dans la phase de réduction, on garde bien le même format.

return key_values

On retourne la liste des couples clé-valeurs

if __name__ == '__main__':

`write_kv_array(read_word_count_lines(sys.stdin))`

Le programme main est le point d'entrée du mapper, on lit les données, les transforme et on les

```
import sys

def write_kv_array(kv_array):
    for kv in kv_array:
        print(f'{kv["key"]}\t{kv["value"]}')

def read_word_count_lines(text):
    key_values = []

    for line in text:
        line = line.strip()
        if not line:
            continue

        parts = line.split('\t')
        if len(parts) != 2:
            continue

        word, count_str = parts[0], parts[1]
        try:
            count = int(count_str)
        except ValueError:
            continue

        key_values.append({'key': 'MAX', 'value': f'{count}\t{word}'})

    return key_values

if __name__ == '__main__':
    write_kv_array(read_word_count_lines(sys.stdin))
```

réécris vers la sortie standard.

Nous allons maintenant implémenter le reducer :

`import sys`

On utilise aussi l'entrée standard.

`def read_kv_array(text):`

`key_values = []`

Création d'une liste qui permet de stocker les couples reçus.

`for line in text:`

`line = line.strip()`

`if not line:`

`continue`

Chaque ligne est nettoyée et on ignore les lignes vides.

`parts = line.split('\t', 1)`

```
if len(parts) != 2:
```

```
    continue
```

On sépare chaque ligne, on sépare la clé de la valeur grâce à la tabulation en limitant à une seule tabulation.

```
    key, value = parts[0], parts[1]
```

```
    key_values.append({'key': key, 'value': value})
```

On stocke les clés et les valeurs dans la liste initialisée au préalable.

```
    return key_values
```

on retourne la liste complète des couples

```
def write_result(word, count):
```

```
    print(f'{word}\t{count}')
```

On affiche le résultat sous la forme attendue.

```
def most_frequent(kv_array):
```

```
    best_count = None
```

```
    best_word = None
```

```
    for kv in kv_array:
```

```
        if kv['key'] != 'MAX':
```

```
            continue
```

On initialise les variables qui stockeront le mot le plus fréquent. Et on prend en compte seules les paires associées à la clé MAX.

```
        parts = kv['value'].split('\t', 1)
```

```
        if len(parts) != 2:
```

```
            continue
```

```
        count_str, word = parts[0], parts[1]
```

La valeur est découpée pour extraire le compteur et le mot.

```
        try:
```

```
            count = int(count_str)
```

```
        except ValueError:
```

```
            continue
```

On convertit le compteur en entier comme dans le mapper

```
        if best_count is None or count > best_count or (count == best_count and word < best_word):
```

```
            best_count = count
```

```
best_word = word
```

```
return best_word, best_count
```

On conserve le mot ayant le plus d'occurrences, si il y a une égalité, on choisit le mot le plus petit.
Puis on retourne le mot le plus fréquent.

```
import sys

def read_kv_array(text):
    key_values = []

    for line in text:
        line = line.strip()
        if not line:
            continue

        parts = line.split('\t', 1)
        if len(parts) != 2:
            continue

        key, value = parts[0], parts[1]
        key_values.append({'key': key, 'value': value})

    return key_values

def write_result(word, count):
    print(f'{word}\t{count}')

def most_frequent(kv_array):
    best_count = None
    best_word = None

    for kv in kv_array:
        if kv['key'] != 'MAX':
            continue

        parts = kv['value'].split('\t', 1)
        if len(parts) != 2:
            continue

        count_str, word = parts[0], parts[1]
        try:
            count = int(count_str)
        except ValueError:
            continue

        if best_count is None or count > best_count or (count == best_count and word < best_word):
            best_count = count
            best_word = word

    return best_word, best_count

if __name__ == '__main__':
    word, count = most_frequent(read_kv_array(sys.stdin))
    if word is not None:
        write_result(word, count)
```

1,1

```
def most_frequent(kv_array):
    best_count = None
    best_word = None

    for kv in kv_array:
        if kv['key'] != 'MAX':
            continue

        parts = kv['value'].split('\t', 1)
        if len(parts) != 2:
            continue

        count_str, word = parts[0], parts[1]
        try:
            count = int(count_str)
        except ValueError:
            continue

        if best_count is None or count > best_count or (count == best_count and word < best_word):
            best_count = count
            best_word = word

    return best_word, best_count

if __name__ == '__main__':
    word, count = most_frequent(read_kv_array(sys.stdin))
    if word is not None:
        write_result(word, count)
```

54,1 Bot

Nous allons maintenant lancer le JOB avec le code qui est donné dans le sujet :

```
mapred streaming -D stream.non.zero.exit.is.failure=false \  
-files most_frequent/mapper.py,most_frequent/reducer.py \  
-input /education/ece_2026_spring_gr01/l.chloe-ece/lab3/word-count \  
-output /education/ece_2026_spring_gr01/l.chloe-ece/lab3/most-frequent \  
-mapper "python3 mapper.py" \  
-reducer "python3 reducer.py"
```

Voici les résultats obtenus :

On voit bien qu'il y a que des succès :

```
26/02/10 08:37:33 INFO streaming.StreamJob: Output directory: /education
/ece_2026_spring_gr01/l.chloe-ece/lab3/most-frequent
[l.chloe-ece@edge-1 03.the-mapreduce-framework_lab-resources]$ hdfs dfs
-ls /education/ece_2026_spring_gr01/l.chloe-ece/lab3/most-frequent
Found 2 items
-rw-r--r--  2 l.chloe-ece ece_2026_spring_gr01          0 2026-02-10 08
:37 /education/ece_2026_spring_gr01/l.chloe-ece/lab3/most-frequent/_SUCC
ESS
-rw-r--r--  2 l.chloe-ece ece_2026_spring_gr01        10 2026-02-10 08
:37 /education/ece_2026_spring_gr01/l.chloe-ece/lab3/most-frequent/part-
00000
[l.chloe-ece@edge-1 03.the-mapreduce-framework_lab-resources]$ hdfs dfs
-cat /education/ece_2026_spring_gr01/l.chloe-ece/lab3/most-frequent/part
-*
the      13604
```

Le mot le plus fréquent est donc THE qui apparaît 13604 fois.